

BridgeAI Technical Audit

Date: May 2, 2026

1. Project Overview

BridgeAI is a CUNY student resource matching application. Its purpose is to collect a student profile from a PDF upload, manual form, or claimed LinkedIn import, then match the student to scholarships, financial aid, public benefits, internships, and campus programs.

The product has two major parts:

- A Next.js frontend for onboarding, dashboard, deadline tracking, login, and chat UI.
- A FastAPI backend that performs PDF text extraction, Claude-based profile extraction, rule filtering, LLM-based ranking/explanations, chatbot replies, resume tailoring, and a placeholder reminder queue.

Important honesty check: the product claims “60+ resources” in the UI, but `resources.json` contains 18 resources.

2. System Architecture

Frontend

- Next.js 16 App Router with React 19.
- `app/page.tsx`: landing page plus floating BridgeBot.
- `app/onboard/page.tsx`: intake UI. Calls backend `/extract-profile` and `/match/ledger`.
- `app/dashboard/page.tsx`: reads matches from `localStorage`, displays fit scores, warnings, cards, saved items, chatbot, and resume curation modal.
- `app/deadlines/page.tsx`: reads matched resources from `localStorage`, sorts deadlines, creates Google Calendar URLs, calls reminder endpoint.
- `app/login/page.tsx`: demo auth only, stores user in `localStorage`.
- `components/navbar.tsx`: navigation and local user state.
- `components/bridgebot-widget.tsx`: landing chatbot client.

Backend

- `main.py` defines a FastAPI app.
- Deployed via Procfile: `uvicorn main:app --host 0.0.0.0 --port $PORT`.
- CORS allows `FRONTEND_URL`, wildcard fallback, and any `*.vercel.app` origin.

Data Layer

- No database.
- Resource catalog is static JSON in `resources.json`.
- User profile, matches, saved resources, reminders, and demo login are stored in browser `localStorage`.
- Backend uses `lru_cache` to cache resource JSON.

Deployment

- Frontend appears intended for Vercel.
- Backend appears intended for Railway.
- `next.config.ts` is empty. Build warns that Next/Turbopack may infer the wrong workspace root because a higher-level lockfile exists. Pinning `turbopack.root` would make builds more reliable.

3. AI Classification

Classification: hybrid rule-based + LLM-powered AI system.

It is not a machine learning system in the training/inference sense. There is no model training, no local ML model, no embeddings, no vector database, no feature store, no evaluation loop, and no learned ranking model.

It is LLM-powered because:

- Claude is called through `anthropic.Anthropic`.
- The configured model is `claude-haiku-4-5-20251001`.
- LLM calls power PDF profile extraction, match explanation/ranking, chat, and resume tailoring.

It is also rule-based because the core eligibility filter is deterministic: GPA, credits, income, max credits, and citizenship are checked directly in `main.py`.

4. AI Components Found

Input Processing

- PDF upload accepted in `/extract-profile`.
- PDF text extracted with `pdfplumber`.
- Manual form input collected in `app/onboard/page.tsx`.
- LinkedIn import is fake: it only validates that the URL contains `linkedin.com`, then submits a hardcoded profile.

Feature Extraction

- LLM extracts `gpa`, `credits`, `major`, `income`, `skills`, `enrollment`, `citizenship`, `first_gen`, `dependents`, and `borough` from text.
- Manual form creates the same profile fields without AI.

Retrieval

- No semantic retrieval.
- No RAG.
- No search index.
- Retrieval is just loading all static resources from JSON.

Scoring / Inference

- Eligibility filtering is rules-based.
- LLM produces `fit_score` and `match_reason`.
- If ranking fails, every match silently defaults to `fit_score = 70`.

Ranking

- Matches are sorted descending by `fit_score`.
- This is LLM ranking, not a validated ranking model.

Explanation

- Explanations are generated as one-sentence `match_reason`.
- Warnings are deterministic, e.g. Excelsior risk and GPA threshold warnings.

ML / LLM Usage

- Claude profile parser.
- Claude ranking/explanation.
- Claude landing bot.
- Claude dashboard bot.
- Claude resume tailoring.

5. AI Pipeline

1. User starts on onboarding.
2. User selects one of three paths: PDF upload, manual form, or LinkedIn URL.
3. PDF path: frontend sends first uploaded PDF only to `/extract-profile`.
4. Backend extracts text with `pdfplumber`.
5. Backend sends first 4,000 characters to Claude.
6. Claude returns JSON.
7. Pydantic validates the student profile.
8. Manual path: frontend builds profile JSON directly.
9. Match path: frontend sends profile to `/match/ledger`.
10. Backend loads `resources.json`.
11. Backend filters by GPA, credits, max credits, income, and citizenship.
12. Backend asks Claude to score/rank candidate resources.
13. Backend returns matches, access score, total value, warnings, and profile.

14. Frontend stores results in `localStorage`.
15. Dashboard reads `localStorage`, displays cards, fit bars, explanation text, deadline states, and saved resources.
16. Chatbot sends profile plus messages to `/chat/bridge-bot`.
17. Resume curation sends resource/job info plus student skills to `/tailor-resume`.
18. Deadline page reads matched resources from `localStorage`; reminder endpoint only prints a message server-side.

6. AI Principles Used

Explainable AI

Present, but shallow. The system gives match reasons and warning messages. There is no traceable explanation of which exact rule passed/failed per resource. There is no confidence calibration or evidence citation.

Decision Systems

Strongest actual component. The eligibility engine is a deterministic decision system over a static ledger. However, missing income is treated as non-disqualifying, which can overmatch students to income-limited resources.

Optimization

Minimal. Sorting by LLM fit score is an optimization-like behavior, but not an optimized objective. There is no constraint solver, no expected-value optimization, and no application-priority optimizer.

NLP

Present through Claude for PDF field extraction, chat, explanations, and resume tailoring.

RAG

Not present. The LLM does not retrieve from a vector store or live source corpus. It receives hardcoded prompt knowledge and a JSON-derived candidate list.

Agent Architecture

Not present. There are no tools, planning loops, memory, autonomous task execution, or multi-step agent workflows. BridgeBot is a stateless chat completion wrapper.

7. API Analysis

Internal Backend Endpoints

- POST `/extract-profile`: PDF to student profile.
- POST `/match/ledger`: profile to eligibility matches.
- POST `/chat/landing-bot`: public chat.
- POST `/chat/bridge-bot`: profile-aware chat.
- GET `/nyc-jobs`: calls NYC Open Data, but no frontend usage was found.
- POST `/tailor-resume`: Claude-generated resume advice.
- POST `/schedule-gmail-reminder`: placeholder only, does not send Gmail.
- GET `/health`: status and AI config info.

External APIs

- Anthropic Claude API via `anthropic`.
- NYC Open Data jobs API at <https://data.cityofnewyork.us/resource/kpav-sd4t.json>.
- Google Calendar URL generation in frontend, not an API integration.
- External resource links to HESC, StudentAid, NYC.gov, BMCC, PTK, Goldman Sachs, Google STEP, Handshake, Indeed.
- No OpenAI, Gemini, Supabase, database, email provider, Gmail API, LinkedIn API, or vector DB integration exists in code.

8. Strengths

- The core product concept is coherent: profile extraction plus eligibility matching plus explanations is a real civic decision-support workflow.
- The backend has a clean small surface: typed Pydantic schemas, explicit endpoints, cached static data, and simple deploy shape.
- The hybrid approach is sensible: deterministic filtering handles hard eligibility constraints, while Claude handles messy language tasks and human-readable explanations.
- The UX is stronger than the backend: onboarding, dashboard cards, deadline tracker, saved resources, chatbot, and resume tailoring create a complete demo flow.
- The static resource schema is practical: each resource has category, value, constraints, deadline, description, and link.

9. Weaknesses

The biggest technical gap: this is not a robust AI eligibility engine yet. It is a demo-grade rule filter plus LLM-generated polish.

Critical issues:

- Missing income overmatches users. Income filtering only runs if `profile.income` is truthy. If income is `None`, students pass all income-limited resources.

- `/match/ledger` swallows all Claude failures, including missing API key. The endpoint can silently return default scores and generic explanations while appearing AI-powered.
- No real LinkedIn extraction. The UI promises extraction, but submits a fixed demo profile.
- No real Gmail reminder. The backend only prints to server logs.
- No authentication. Login stores user info in `localStorage`; it is not identity, security, or persistence.
- Sensitive student data is stored in browser `localStorage` and sent to Anthropic without visible consent, redaction, retention policy, or audit logging.
- CORS defaults to `*`, which is too permissive for a system handling transcripts and income data.
- No database, so no durable user state, audit history, application status, reminders, or analytics.
- No tests for eligibility rules, API schemas, malformed LLM outputs, or frontend flows.
- No RAG or source freshness. Benefit rules and deadlines are hardcoded in JSON and prompts.
- No validation that Claude’s generated `fit_score` is bounded 50-100 despite the prompt asking for it.
- `access_score` is just `matches / total_resources * 100`, not a meaningful student access measure.
- The README claims a pipeline but stops at “How It Works”; documentation is incomplete.

10. Recommendations

Highest Priority Upgrades

- Fix eligibility semantics: distinguish eligible, ineligible, unknown, and needs verification. Missing income should not equal eligible.
- Return rule-level explanations per resource: passed rules, failed rules, unknown evidence, source URL, last verified date.
- Replace silent AI fallback in `/match/ledger` with explicit `ai_configured` behavior or a clear deterministic-only mode.
- Add real persistence: Postgres/Supabase/Firebase for profiles, matches, saved resources, reminders, and audit events.
- Add real auth: Supabase Auth, Clerk, Auth.js, or CUNY SSO if available.
- Add privacy controls: file size limits, PII redaction, consent before sending documents to Claude, retention policy, and deletion.
- Make reminders real: SendGrid, Resend, Gmail API, or scheduled worker.
- Replace fake LinkedIn import with either a removed UI path or a real parser/import flow.

AI Upgrades

- Add embeddings and RAG over official program pages, PDFs, and scraped policy documents.
- Store source chunks with program rules and citations.
- Use an extraction pipeline: document OCR/text extraction, structured field extraction, confidence scoring, user confirmation.
- Use deterministic rule evaluation for final eligibility, not LLM judgment.
- Use the LLM only for explanation, summarization, missing-info questions, and action plans.
- Add a ranking model or transparent scoring formula based on value, urgency, eligibility confidence, deadline proximity, and student goals.
- Add an agent only after the basics are stable: an application assistant that can build checklists, draft forms, gather missing documents, and track next steps.

Verification Performed

- `npm run lint`: passed.
- `npm run build`: passed outside sandbox.
- `python3 -m py_compile main.py`: passed.